SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5111

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS: **Total Marks: 50**

Annexure A

Code of Conduct:

I Kala Gowri Shankar (192325109) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A
Debugging & Optimisation Task

2. Performance Analysis of Block Cipher Modes of Operation

A Python script implements AES-like block cipher modes (ECB, CBC, CFB, OFB) but suffers from high latency and memory inefficiency.

1. Debug incorrect padding and IV handling causing decryption failures.
2. Refactor the code to minimize redundant computations and memory usage.
3. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

A. Debug Incorrect Padding and IV Handling Causing Decryption Failures

Problems Identified

1. Incorrect Padding

- AES works with fixed-size blocks (16 bytes).
- If the plaintext size is not a multiple of the block size, encryption and decryption may fail.
- Incorrect padding removal can produce corrupted output.

Solution:

- Implement standard PKCS#7 padding.
- Validate padding length during decryption.
- Remove padding only after successful decryption.

2. Improper Initialization Vector (IV) Handling

- The same IV should not be reused for multiple encryption operations.
- Using different IVs during encryption and decryption causes incorrect plaintext recovery.

Solution:

- Generate a random IV for every encryption process.
- Store or transmit the IV along with the encrypted data.
- Use the same IV during decryption.

3. Block Size Mismatch

- Input data must be divided into correct block sizes.
- Incorrect block handling leads to encryption and decryption errors.

Solution:

- Check input length before processing.
- Apply proper block management techniques.

B. Refactor the Code to Minimize Redundant Computations and Memory Usage

Optimization Techniques

4. Process Data Block by Block — avoid loading the complete file into memory; encrypt and decrypt one block at a time.
5. Reuse Memory Buffers — use existing memory locations instead of creating new objects repeatedly, reducing memory consumption.
6. Generate Keys Once — perform key expansion only one time and reuse the generated round keys for all blocks.
7. Remove Unnecessary Operations — avoid repeated calculations inside loops and use optimized functions for encryption and decryption.
8. Improve Data Handling — use streaming methods for large data processing and reduce unnecessary copying of ciphertext and plaintext.

Result of Optimization

Before Optimization

- High memory usage.
- Repeated calculations.
- Increased execution time.

After Optimization

- Lower memory consumption.
- Faster encryption and decryption.
- Better performance for large data.

C. Compare and Optimize the Implementation to Determine the Most Efficient Mode

Mode	Advantages	Disadvantages
ECB	Simple and fast; supports parallel processing	Does not hide patterns; less secure
CBC	Provides better security using an IV	Slower, because each block depends on the previous block
CFB	Suitable for streaming data	Limited parallel processing
OFB	Fast, low memory usage, no error propagation	Requires a unique IV

Mode Analysis

ECB Mode

- Fastest mode.
- Not recommended for secure communication because repeated plaintext blocks create repeated ciphertext patterns.

CBC Mode

- Provides better security than ECB.
- Requires correct padding and IV management.

CFB Mode

- Useful for real-time communication.
- Works efficiently with continuous data streams.

OFB Mode

- Provides fast encryption and decryption.
- Does not propagate transmission errors.
- Requires less memory.

Conclusion

For real-time secure communication, OFB mode is the most efficient choice because it provides:

- High speed
- Low memory consumption
- No error propagation

3. Debugging RSA Cryptosystem Implementation

A Python-based RSA encryption system fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

1. Identify issues in prime generation, key generation, and modular exponentiation.
2. Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).
3. Evaluate the trade-offs between key size, security, and computational performance.

A. Identify Issues in Prime Generation, Key Generation, and Modular Exponentiation

1. Problems in Prime Generation

- Small prime numbers reduce RSA security.
- Weak random number generation can create predictable keys.
- Invalid prime numbers may be selected.

Solution:

- Generate large random prime numbers.
- Use the Miller-Rabin primality test to verify primes.
- Use secure random number generators.

2. Problems in Key Generation

- Incorrect selection of the public key value.
- Private key calculation errors.
- Public and private keys may not satisfy RSA mathematical requirements.

Solution:

- Select a suitable public exponent, such as 65537.
- Calculate the private key using the Extended Euclidean Algorithm.
- Ensure generated keys are mathematically valid.

3. Problems in Modular Exponentiation

- Direct calculation of large powers requires huge memory.
- It causes overflow and slow execution.

Solution:

- Use modular exponentiation techniques.
- Apply the Square-and-Multiply algorithm for faster computation.

B. Optimize the RSA Algorithm

1. Fast Modular Exponentiation

- Uses the Square-and-Multiply method.
- Reduces the number of multiplication operations.
- Improves encryption and decryption speed.

Advantages:

- Faster computation.
- Less memory usage.
- Suitable for large RSA keys.

2. Chinese Remainder Theorem (CRT)

CRT improves RSA decryption performance. Instead of performing one large calculation:

- The operation is divided into smaller calculations using prime numbers p and q.
- The results are combined to obtain the final plaintext.

Advantages:

- Decryption becomes approximately 3 to 4 times faster.
- Reduces computational workload.

3. Efficient Prime Testing

- Use the Miller-Rabin algorithm instead of simple division methods.
- Generates secure primes quickly.

C. Trade-off Between Key Size, Security, and Performance

RSA Key Size	Security Level	Performance
1024-bit	Low security	Very fast
2048-bit	High security	Good performance
3072-bit	Very high security	Slower
4096-bit	Maximum security	Requires more computation

Analysis

Small Key Size

- Faster encryption and decryption.
- Less secure against modern attacks.

Large Key Size

- Provides stronger security.
- Requires more processing power and memory.

Conclusion

A 2048-bit RSA key provides the best balance between security and performance. Using Fast Modular Exponentiation and the Chinese Remainder Theorem (CRT) improves RSA efficiency while maintaining strong security.

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation